

ASSIGNMENT 3

Instructions

- Create separate `.js` files.
 - Use `"use strict"` wherever applicable.
 - Add comments explaining logic, hoisting effects, and debug observations.
 - Avoid using external libraries.
 - All output must be visible via `console.log()`.
-

Q1 – Dynamic Data Parser

You receive mixed API data:

```
const apiData = ["25", "true", "false", "NaN", " ", "100px", "3.14", null, undefined];
```

Tasks:

1. Convert each value into a **Number**, **Boolean**, and **String** form.
 2. Skip invalid numbers (`NaN`, `" "`, `"100px"`) but log them separately.
 3. Build two arrays — one for **valid numeric data**, one for **invalid**.
 4. Print a detailed report using loops and conditional formatting.
-

Q2 – Employee Bonus Calculator

Given employee data:

```
const employees = [  
  { name: "Amit", salary: "45000", years: "5" },  
  { name: "Sara", salary: "38000", years: "2" },  
  { name: "Kiran", salary: "52000", years: "7" }  
];
```

Tasks:

1. Convert salary and years to numbers.
 2. Calculate bonus = (salary * 0.1) if years > 3 else (salary * 0.05).
 3. Apply strict validation using "use strict" (no implicit globals).
 4. Use template strings to print each employee's formatted details.
 5. Add try...catch to handle conversion or missing property errors.
-

Q3 – Transaction Validator

```
const transactions = [  
  { id: 1, amount: 2000 },  
  { id: 2, amount: -500 },  
  { id: 3 },  
  null  
];
```

Tasks:

1. Loop through each transaction.
2. Throw custom errors for:

- Negative amount
 - Missing amount or id
 - Null entries
3. Catch and categorize each error type in arrays (`invalid`, `valid`).
 4. Print final reports with counts of successful and failed transactions.
 5. Use a breakpoint to watch variable states.

Q4 – Debugging Mystery

Start with:

```
"use strict";
function showMessage() {
  greeting = "Welcome"; // undeclared
  console.log(greeting);
}
showMessage();
```

Tasks:

1. Identify why this throws an error under strict mode.
2. Fix it and explain how scope declaration rules changed behavior.
3. Add a watch variable and observe the call stack in VS Code's debugger.

Q5 – Hoisting Lab: The Sequence Trap

Predict, then run:

```
console.log(score);
announce();
var score = 50;
function announce() { console.log("Game started"); }
let status = "ready";
startGame();
function startGame() {
  console.log(status);
}
```

Tasks:

1. Explain each hoisted element's memory state.
 2. Fix the code to run properly.
 3. Rewrite using arrow functions to compare hoisting differences.
-

Q6 – Pyramid Pattern Generator

Generate:

```
*
* *
* * *
* * * *
```

Then:

1. Replace `let` with `var` and observe changes in output.
2. Debug step-by-step to track variable re-use.
3. Add an outer loop limit controlled by user input (default = 5).
4. Use `"use strict"` to catch undeclared loop variables.

Q7 – Smart Calculator

```
const operations = ["add", "divide", "power", "root", "subtract"];
const num1 = 25, num2 = 0;
```

Tasks:

1. Use a switch to handle each operation.
2. Add custom error handling for divide by 0 and root of negative number.
3. If operation not recognized, throw `InvalidOperationError`.
4. Wrap in `try...catch` and print results in a formatted summary.

Q8 – Strict Mode Showdown

```
function demo(a, a) {
  total = 10;
  delete total;
}
demo(5, 10);
```

Tasks:

1. Run this with and without `"use strict"`.
 2. Record the errors and differences in behavior.
 3. Explain why strict mode makes these operations illegal.
 4. Show a correct version using ES6 rules.
-

Q9 – JSON Audit

```
const rawData = [  
  '{"user":"Alex","age":25}',  
  '{"id":2}',  
  '{invalid}',  
  '{"user":"Mina","age":"22"}'  
];
```

Tasks:

1. Parse each entry in a loop using `try...catch`.
 2. Detect missing keys (user, age) and invalid JSON.
 3. Push valid entries to a clean array; log errors with line numbers.
 4. Debug step-by-step to see how control flows on error.
 5. Add bonus: convert age to Number and filter under-18 users.
-

Q10 – Nested Hoisting and Closures

```
function outer() {  
  console.log(count);  
  var count = 5;  
  function inner() {  
    console.log(count);  
    var count = 10;  
  }  
  inner();  
}  
outer();
```

Tasks:

1. Predict and explain output.
2. Show how hoisting creates separate memory contexts.
3. Convert the inner function to an arrow and note behavior changes.
4. Add a debug session and explain the call stack flow.